CSE 4165: Web Programming — Code Syntexes, Templates & SQL Queries

1. TEMPLATE: JavaScript (template-js.html)

```
<body>
    <input type="text" id="input1">
    <button onclick="pass()" id="button1">Submit</button>

    <p id="output1"></p>
    <p id="output2"></p>

    <script>
        function pass() {
            let input1 = document.getElementById('input1').value

            // if input is a number
            let inputNum = Number(input1);

            document.getElementById('output1').innerText = input1;
            document.getElementById('output2').innerText = "Hello JS";
        }
    </script>
</body>
```

2. TEMPLATE: PHP Form Handling (template-php.php)

```
<body>
    <form action="" method="POST">
        <input type="text" name="input1">
        <input type="text" name="input2">
        <input type="text" name="input3">

        <button name="submit">Submit</button>
    </form>

    <?php
        if(isset($_POST['submit'])) {
            $input1 = (int)$_POST['input1'];
            $input2 = (int)$_POST['input2'];
            $input3 = (int)$_POST['input3'];

        }
    ?>
</body>
```

3. TEMPLATE: MySQLi (template-mysql.php)

```
<body>
    <?php
        $conn = new mysqli("127.0.0.1", "root", "", "db_name", 3306);
        if ($conn->connect_error) die("Connection failed: " . $conn->
     connect_error);
        else echo "<h2>Database connected</h2>";

        // SELECT query
        $query1 = "write_query";
        $store1 = $conn->query($query1);
        while($show = $store1->fetch_assoc()) {
            echo "". $show["col_name"]. " ". $show["col_name"]. "<br>";
        }

        // UPDATE query
        $conn->query("UPDATE table_name SET col_name = 'new_val' WHERE id
      = 1");
    ?>
</body>
```

4. MYSQL EXAM QUERIES

Quick Rules: `LIKE: 'A%'` starts | `'%A'` ends | `'%A%'` contains | `'_a%'` 2nd a. `BETWEEN x AND y` (both ends included). `ASC` (low→high) | `DESC` (high→low). `WHERE` filters rows; `HAVING` filters groups. Math: `+10%` → `*1.10` | `-10%` → `*0.90`.

• 1. SELECT, Filter & Sorting

```
-- Basic SELECT, Condition & Sort (ASC / DESC)
SELECT BookTitle, Price FROM book_info
WHERE Category = 'Programming'
ORDER BY Price DESC;

-- Complex WHERE: AND, OR, LIKE, BETWEEN
SELECT * FROM book_info
WHERE BookTitle LIKE '%Web%'
   OR (Category = 'Programming' AND Price BETWEEN 450 AND 600);
```

• 2. Aggregation & GROUP BY

```
-- Total count of entire table
SELECT COUNT(*) AS TotalBooks FROM book_info;

-- COUNT, AVG, SUM grouped by Category
SELECT Category, COUNT(*) AS N, AVG(Price) AS AvgPrice,
       SUM(Price) AS TotalPrice
FROM book_info
GROUP BY Category;
```

• 3. GROUP BY with HAVING & ORDER BY

```
-- HAVING on COUNT + ORDER BY alias
SELECT Category, COUNT(*) AS N
FROM book_info
GROUP BY Category
HAVING COUNT(*) > 1
ORDER BY N DESC;

-- HAVING on AVG/SUM + ORDER BY
SELECT Category, AVG(Price) AS AvgPrice, SUM(Price) AS TotalPrice
FROM book_info
GROUP BY Category
HAVING AVG(Price) > 300
ORDER BY AvgPrice DESC;
```

• 4. Total Net Worth / Calculated Math

```
-- Entire table: multiply price * stock & sum all
SELECT SUM(Price * Stock) AS TotalWorth FROM book_info;

-- Grouped sum of product with HAVING
SELECT Category, SUM(Price * Stock) AS TotalWorth
FROM book_info
GROUP BY Category HAVING TotalWorth > 5000;
```

• 5. UPDATE with Math & Conditions

```
-- Update multiple cols with math (-10% / +10%)
UPDATE book_info
SET Price = Price * 0.90, Stock = Stock + 2
WHERE Category = 'Language';

-- Ceiling check: bonus only if resulting stock <= 15 & !=
UPDATE book_info
SET Stock = Stock + 3
WHERE Category != 'Business' AND (Stock + 3) <= 15;
```

• 6. Subqueries (Single Table)

```
-- Filter above overall table average
SELECT BookTitle, Price FROM book_info
WHERE Price > (SELECT AVG(Price) FROM book_info);

-- Correlated: filter above own category's average
SELECT BookTitle, Price FROM book_info b1
WHERE Price > (SELECT AVG(Price) FROM book_info b2
               WHERE b2.Category = b1.Category);
```

CSE 4165: Web Programming — Essential JS & PHP Exam Syntax

5. JAVASCRIPT: Essential Exam Patterns

• 1. Random Number in Range [min, max]

```
let secret = Math.floor(Math.random() * (max - min + 1)) + min;
// Example: Random number between 500 and 5000
let secret = Math.floor(Math.random() * (5000 - 500 + 1)) + 500;
```

• 2. Loop String: Check Capital, Small & Digit

```
let hasUpper = false, hasLower = false, hasDigit = false;
for (let i = 0; i < str.length; i++) {
  let ch = str[i];
  if (ch >= 'A' && ch <= 'Z') hasUpper = true;
  else if (ch >= 'a' && ch <= 'z') hasLower = true;
  else if (ch >= '0' && ch <= '9') hasDigit = true;
}
```

• 3. History Array: Push, Loop Sum & Average

```
let history = []; // Keep track of inputs
history.push(val);
let sum = 0;
for (let i = 0; i < history.length; i++) {
  sum += history[i];
}
let avg = sum / history.length;
```

• 4. Delimited String Splitting (e.g. "Math-5, Eng-4")

```
let items = inputStr.split(',');
let totalHours = 0;
for (let item of items) {
  let parts = item.trim().split('-');
  let name = parts[0].trim();
  let hours = Number(parts[1].trim());
  totalHours += hours;
}
```

• 5. Guess Limit: Counter & Disable Inputs

```
attempts++;
if (guess === secret) {
  document.getElementById('output1').innerText = "Correct!";
  document.getElementById('button1').disabled = true;
  document.getElementById('input1').disabled = true;
} else if (attempts >= 5) {
  document.getElementById('output1').innerText = "Limit reached!";
  document.getElementById('button1').disabled = true;
  document.getElementById('input1').disabled = true;
}
```

• 6. Essential JS Helpers (Quick Scan)

```
Math.abs(a - b);        // Absolute positive difference |a - b|
num.toFixed(2);         // Format float to 2 decimal places
str.trim(); str.length; // Trim spaces & get character count
document.getElementById('input1').value = ""; // Clear input box
let top2 = [ct1, ct2, ct3].sort((a, b) => b - a); // Descending
```

6. PHP: Essential Exam Patterns

• 1. Ceil Allocation Units & Wasted Cost

```
// Units needed (cannot book partial, e.g. 5.1 -> 6)
$needed = ceil($total_demand / $unit_capacity);

// Unused / empty capacity:
$empty = ($needed * $unit_capacity) - $total_demand;

// Wasted money on unused units:
$wasted_money = $empty * $cost_per_seat;
```

• 2. Best 2 of 3 CT Marks (rsort)

```
$scores = [$ct1, $ct2, $ct3];
rsort($scores); // Sorts descending: [0] and [1] are best 2
$best2_avg = ($scores[0] + $scores[1]) / 2;
```

• 3. Target Comparison & Absolute Difference

```
$diff = abs($achieved - $target);
if ($achieved > $target) {
    echo "Above target by " . $diff;
} elseif ($achieved < $target) {
    echo "Below target by " . $diff;
} else {
    echo "Target met exactly (0 difference)";
}
```

• 4. Delimited String Splitting (explode) & Foreach

```
$items = explode(',', $input_str); // Split by comma
$total = 0;
foreach ($items as $item) {
    $total += (float)trim($item);
}
```

• 5. Essential PHP Built-Ins (Quick Scan)

```
ceil($n);        // Round UP to next integer (2.1 -> 3)
floor($n);       // Round DOWN to integer (2.9 -> 2)
round($n, 2);    // Round to 2 decimal places
abs($n);         // Absolute positive value (|-5| -> 5)
min($a, $b); max($a, $b); // Minimum / Maximum of numbers
rsort($arr);     // Sort array in DESCENDING order
count($arr);     // Count elements in array
```

• 6. Common Exam Traps (Quick Checklist)

```
// 1. In $_POST['key'], 'key' matches HTML name="..." (NOT id)
// 2. In PHP string concat is '.' NOT '+' ($a . $b)
// 3. Always cast: $num = (int)$_POST['key']; to prevent string concat
// 4. SQL column names DO NOT take single quotes ('col' is wrong)
```